

RAG Pipeline Step 4: Advanced Query Translation

This document covers advanced techniques that sit between the user's input and the Retrieval step.

1. Overall Definition

Query Translation (also called Query Analysis or Query Transformation) is an advanced pre-retrieval process. It is built on a single, critical idea: the user's initial query is often *not* the best query to send to the vector database.

- **Problem:** The user's query might be ambiguous ("What is it?"), too complex ("Compare X and Y"), or use different words than the source documents ("PTO" vs. "annual leave").
- **Solution:** Use an LLM to "translate" the user's raw query into one or more new queries that are optimized for retrieval. This step acts as an intelligent "fixer" to ensure the retrieval step gets the best possible context.

Here are the five key techniques you identified:

2. Multi-Query

- **What is it?** This is the process of taking a single user query and generating *multiple* variations of it from different perspectives.
- **When to use it:** This is the best technique for tackling **ambiguity** and **mismatched terminology**. If a user asks, "How do I get time off?", a multi-query approach might generate:
 - "What is the company policy for annual leave?"
 - "How to request PTO?"
 - "Steps to book a vacation day."
- **How it works (Process):**
 1. The user's original query is passed to an LLM.
 2. The LLM, guided by a prompt, generates 3-5 new, related queries.
 3. The system retrieves documents for *all* of these queries (plus the original).
 4. All retrieved documents are combined, duplicates are removed, and the full set of unique documents is passed to the generation step.
- **Key LangChain Tool:** MultiQueryRetriever. This is a built-in retriever that automates this entire flow. You provide it with an LLM and a vector store retriever, and it handles the query generation and document union automatically.

3. RAG Fusion

- **What is it?** RAG Fusion is the *evolution* of Multi-Query. It also generates multiple queries, but it adds a crucial final step: **intelligent re-ranking**.
- **When to use it:** Use this when Multi-Query returns *too many* documents, and many of them are low-quality or irrelevant. RAG Fusion helps find the "best of the best" from all the different result sets.
- **How it works (Process):**
 1. It follows the exact same first steps as Multi-Query (generate 3-5 new queries, retrieve documents for each).
 2. It then applies a **Reciprocal Rank Fusion (RRF)** algorithm to all the results.
 3. **RRF Logic:** This algorithm re-ranks the combined documents. Documents that appear in the top results for *multiple* queries get a much higher "fusion" score than documents that only appear once.
 4. The system returns a single, re-ranked list of the highest-scoring documents, which are considered the most relevant.
- **Key LangChain Tool:** This is a *pattern* rather than a single tool. You implement it by combining a MultiQueryRetriever (to get the multiple lists) and then manually applying an RRF re-ranking function to the results before generation.

4. Decomposition

- **What is it?** This is the process of breaking one *complex* query into multiple *simpler, distinct sub-queries*.
- **When to use it:** This is for **multi-part** or **comparison** questions. A simple similarity search will fail on a query like, "How is RAG Fusion different from the Multi-Query Retriever, and which one is better for high-ambiguity queries?"
- **How it works (Process):**
 1. The user's complex query is passed to an LLM.
 2. The LLM (often guided by a few-shot prompt) breaks it down into a list of simpler questions. For the example above:
 - "What is RAG Fusion?"
 - "What is the Multi-Query Retriever?"
 - "What are the differences between RAG Fusion and Multi-Query Retriever?"
 - "When to use RAG Fusion for ambiguity?"
 3. The system retrieves documents for *each* of these sub-queries.
 4. All retrieved context is combined and passed to the *final* LLM, which now has all the building blocks to synthesize a complete answer.
- **Key LangChain Tool:** This is an advanced pattern. It is most often implemented using **LangGraph** (for complex, stateful flows) or a custom chain that uses an LLM with a **Pydantic Output Parser** to force the LLM to return a list[str] of sub-queries.

5. Step-Back Prompting

- **What is it?** This is the technique you correctly identified as "abstracting." It involves taking a very *specific* query and also generating a more *general*, high-level "step-back" query.
- **When to use it:** Use this when the user's query is **too specific** or detail-oriented. A query like, "Which engineer wrote the `create_retrieval_chain` function in LangChain?" might fail. The step-back query, "What is the `create_retrieval_chain` function in LangChain and who are its contributors?", is more likely to retrieve a general document that *contains* the specific answer.
- **How it works (Process):**
 1. The user's specific query is passed to an LLM.
 2. This LLM uses a **Few-Shot Prompt** (this is what you were referring to!) to *teach it* how to "step back." The prompt includes 3-5 examples of [Specific Query] -> [General Query].
 3. The system retrieves documents for *both* the original, specific query and the new, general query.
 4. The combined context is passed to the final LLM, which uses the general context to find the specific answer.
- **Key LangChain Tool:** This is a pattern you build, not a single off-the-shelf tool. The core component is a ChatPromptTemplate that incorporates **Few-Shot Prompting** (e.g., using FewShotChatMessagePromptTemplate) to guide the LLM into generating the step-back query.

6. HyDE (Hypothetical Document Embeddings)

- **What is it?** This is a clever and counter-intuitive technique. Instead of embedding the user's *query*, you first ask an LLM to *generate a fake, hypothetical answer* (a hallucination), and then you embed *that answer* to find real documents.
- **When to use it:** This is for queries where the *question* is semantically very different from the *answer*.
 - **Query:** "What is the company policy on sick leave for part-time employees?"
 - **Hypothetical Answer:** "Part-time employees at our company are eligible for 10 days of paid sick leave per year, accrued on a pro-rata basis..."
 - The hypothetical answer is semantically much "closer" in the vector space to the *real* policy document than the user's question was.
- **How it works (Process):**
 1. The user's query is passed to an LLM (with a simple "write an answer to this" prompt).
 2. The LLM generates a *hypothetical* document.
 3. This hypothetical document is passed to the embedding model.
 4. The resulting vector is used to perform the similarity search against the *real* documents.
- **Key LangChain Tool:** HydeRetriever. This is a built-in retriever that wraps your LLM and vector store to automate this entire "generate, then embed, then retrieve" flow.